

Einsendeaufgabe - REF -E1: Refactoring

1 Einleitung

Im Rahmen dieser Einsendeaufgabe wurde das Thema Refactoring anhand des Refactoring-Katalogs von Martin Fowler untersucht. Ziel war es, ausgewählte Refactorings kennenzulernen, deren Nutzen zu bewerten und praktische Erfahrungen mit der Verbesserung von Quellcode zu sammeln.

Refactoring bezeichnet die strukturierte Überarbeitung von bestehendem Quellcode, ohne dessen fachliche Funktionalität zu verändern. Ziel ist es, die Lesbarkeit, Wartbarkeit und Erweiterbarkeit eines Programms zu verbessern.

Für diese Ausarbeitung wurden verschiedene Refactorings aus dem Fowler-Katalog betrachtet. Zusätzlich wurde ein eigenes Refactoring-Beispiel erstellt und dokumentiert. Abschließend wurde untersucht, inwieweit Generative AI bei Refactoring-Aufgaben unterstützen kann und welche Erfahrungen dabei gemacht wurden.

2 Refactoring nach Martin Fowler

Als Grundlage wurde der Refactoring-Katalog von Martin Fowler verwendet: <https://www.refactoring.com/catalog>

2.1 Extract Function

Das Refactoring „Extract Function“ (früher „Extract Method“) gehört zu den bekanntesten Refactorings aus dem Katalog von Martin Fowler. Dabei werden zusammengehörige Anweisungen aus einer bestehenden Funktion in eine neue, eigenständige Funktion ausgelagert.

Dieses Refactoring verbessert die Lesbarkeit des Quellcodes erheblich. Lange Funktionen werden übersichtlicher und einzelne Aufgaben können klar voneinander getrennt werden. Darüber hinaus erhöht sich die Wiederverwendbarkeit des Codes.

Besonders interessant erscheint dieses Refactoring, da es bereits mit geringem Aufwand zu einer deutlichen Verbesserung der Codequalität führen kann.

2.2 Extract Class

Beim Refactoring „Extract Class“ werden Eigenschaften und Methoden aus einer sehr umfangreichen Klasse in eine neue Klasse ausgelagert.

Dadurch werden Verantwortlichkeiten klarer getrennt und die Komplexität einzelner Klassen reduziert. Das Refactoring unterstützt außerdem das Prinzip der einzelnen Verantwortlichkeit (Single Responsibility Principle).

Dieses Refactoring erscheint besonders sinnvoll, wenn Klassen im Laufe der Entwicklung immer größer werden und unterschiedliche Aufgaben übernehmen.

2.3 Replace Conditional with Polymorphism

Das Refactoring „Replace Conditional with Polymorphism“ ersetzt umfangreiche if- oder switch-Konstruktionen durch Vererbung und Polymorphismus.

Dieses Refactoring war zunächst schwer nachvollziehbar, da für kleinere Programme häufig kein unmittelbarer Vorteil erkennbar ist. Der Nutzen zeigt sich vor allem bei größeren objektorientierten Anwendungen mit vielen unterschiedlichen Varianten eines Verhaltens.

Aus diesem Grund erscheint dieses Refactoring deutlich komplexer als beispielsweise Extract Function oder Extract Class.

3 Verwendete IDE und unterstützte Refactorings

Für die praktische Bearbeitung wurde Visual Studio Code (VS Code) verwendet. Visual Studio Code gehört zu den am weitesten verbreiteten Entwicklungsumgebungen und bietet zahlreiche Funktionen zur Unterstützung von Refactorings.

Insbesondere bei Java unterstützt Visual Studio Code über entsprechende Erweiterungen verschiedene Refactoring-Funktionen. Zu den unterstützten Refactorings gehören unter anderem:

- Rename Symbol (Umbenennen von Variablen, Funktionen oder Klassen)
- Extract Method / Extract Function
- Move File
- Organize Imports
- Extract Variable
- Extract Constant

Die integrierten Refactoring-Funktionen erleichtern die Umstrukturierung von Quellcode erheblich und reduzieren das Risiko von Fehlern bei manuellen Änderungen.

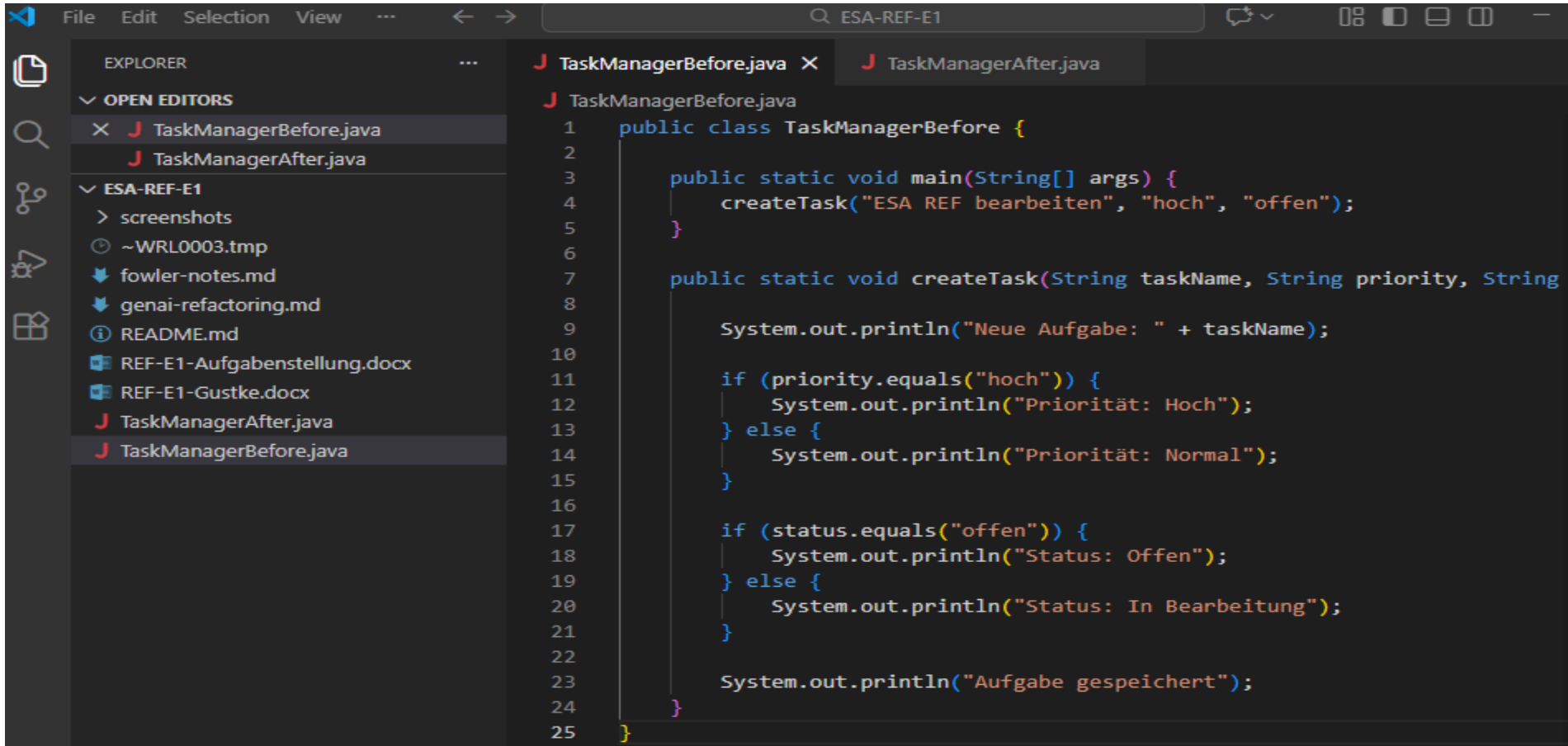
Besonders hilfreich ist dabei die automatische Aktualisierung aller Verwendungsstellen einer Variable oder Funktion. Dadurch können Änderungen schneller und sicherer durchgeführt werden.

4 Eigenes Refactoring-Beispiel

4.1 Ausgangssituation

Für das eigene Refactoring-Beispiel wurde ein vereinfachter Ausschnitt aus einem Smart Task Manager verwendet. Die Methode erstellt eine Aufgabe, prüft deren Priorität, prüft den Bearbeitungsstatus und gibt anschließend eine Speicherbestätigung aus.

Vorher



```
1 public class TaskManagerBefore {
2
3     public static void main(String[] args) {
4         createTask("ESA REF bearbeiten", "hoch", "offen");
5     }
6
7     public static void createTask(String taskName, String priority, String
8
9         System.out.println("Neue Aufgabe: " + taskName);
10
11         if (priority.equals("hoch")) {
12             System.out.println("Priorität: Hoch");
13         } else {
14             System.out.println("Priorität: Normal");
15         }
16
17         if (status.equals("offen")) {
18             System.out.println("Status: Offen");
19         } else {
20             System.out.println("Status: In Bearbeitung");
21         }
22
23         System.out.println("Aufgabe gespeichert");
24     }
25 }
```

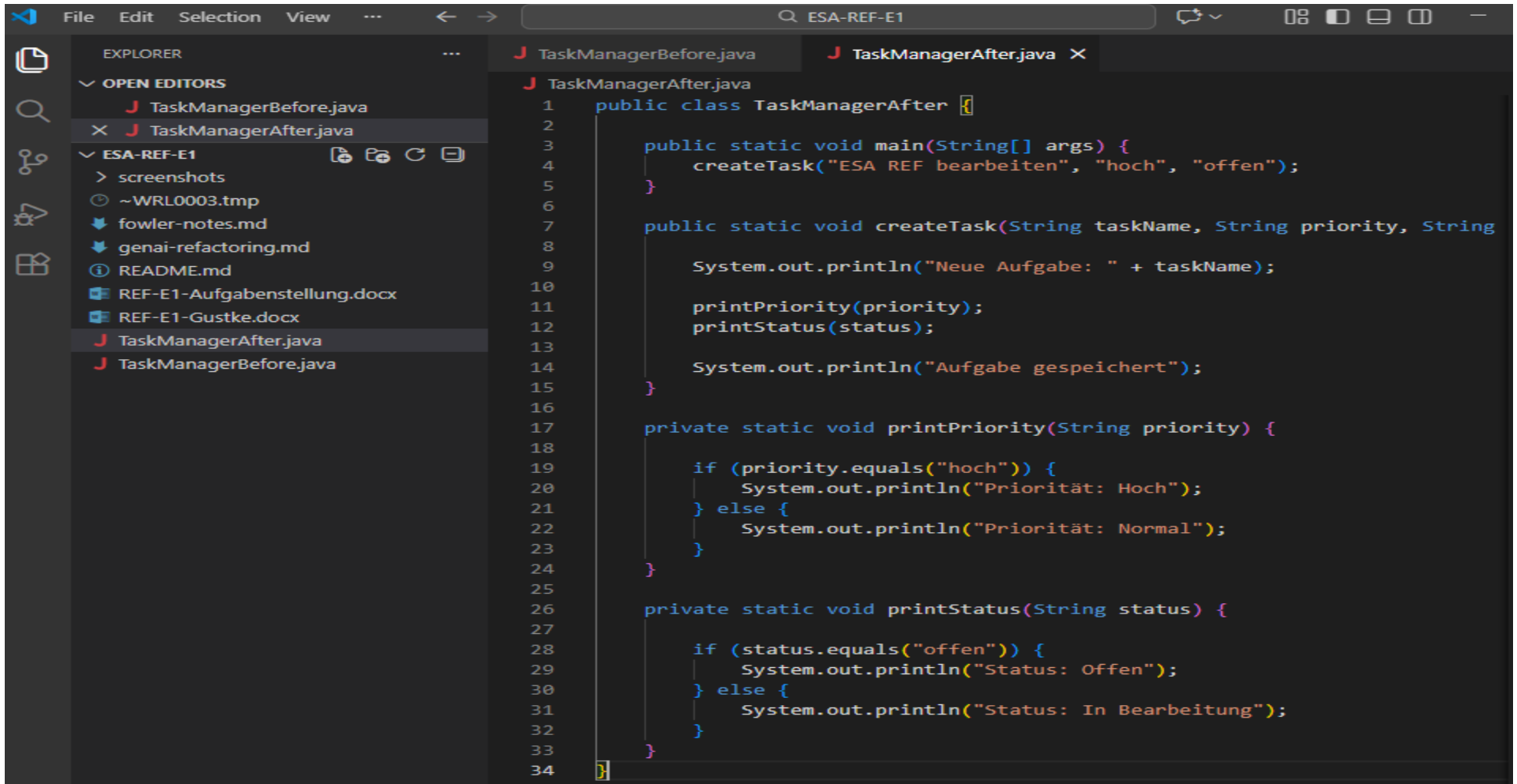
Abbildung: Ausgangscode vor dem Refactoring. Die Methode createTask enthält die Aufgaben Erstellung, Prioritätsprüfung und Statusprüfung.

Der Code funktioniert korrekt, allerdings enthält die Methode createTask mehrere unterschiedliche Verantwortlichkeiten. Neben der Erstellung der Aufgabe werden sowohl die Priorität als auch der Status ausgewertet. Dadurch wird die Methode länger und schwerer wartbar.

4.2 Refactoring

Zur Verbesserung der Lesbarkeit und Wartbarkeit wurden die Prioritätsprüfung und die Statusprüfung in eigene Methoden ausgelagert.

Nachher



```
1 public class TaskManagerAfter {
2
3     public static void main(String[] args) {
4         createTask("ESA REF bearbeiten", "hoch", "offen");
5     }
6
7     public static void createTask(String taskName, String priority, String
8
9         System.out.println("Neue Aufgabe: " + taskName);
10
11         printPriority(priority);
12         printStatus(status);
13
14         System.out.println("Aufgabe gespeichert");
15     }
16
17     private static void printPriority(String priority) {
18
19         if (priority.equals("hoch")) {
20             System.out.println("Priorität: Hoch");
21         } else {
22             System.out.println("Priorität: Normal");
23         }
24     }
25
26     private static void printStatus(String status) {
27
28         if (status.equals("offen")) {
29             System.out.println("Status: Offen");
30         } else {
31             System.out.println("Status: In Bearbeitung");
32         }
33     }
34 }
```

Abbildung: Quellcode nach dem Refactoring mittels Extract Method. Die Prüfungen für Priorität und Status wurden in eigene Methoden ausgelagert.

Durch die Auslagerung der Prioritäts- und Statusprüfung ist die Methode `createTask` deutlich übersichtlicher geworden. Die einzelnen Aufgaben sind nun klar voneinander getrennt. Die neuen Methoden können zudem einfacher wiederverwendet und getestet werden.

4.3 Verwendetes Fowler-Refactoring

Bei diesem Beispiel wurde das Fowler-Refactoring „Extract Function“ verwendet. In Java wird dieses Refactoring häufig als „Extract Method“ bezeichnet.

Dabei werden zusammengehörige Codeabschnitte aus einer bestehenden Methode in eine neue Methode ausgelagert. Ziel ist eine bessere Strukturierung und Lesbarkeit des Quellcodes.

4.4 Bewertung

Das Refactoring führte zu einer klareren Aufteilung der Verantwortlichkeiten innerhalb des Programms. Die Methode `createTask` konzentriert sich nun auf den eigentlichen Ablauf der Aufgabenerstellung, während die Auswertung von Priorität und Status in separaten Methoden erfolgt.

Im Vergleich zur ursprünglichen Version ist der Quellcode übersichtlicher, besser wartbar und leichter erweiterbar. Dieses Beispiel zeigt, dass bereits relativ kleine Refactorings die Qualität von Software deutlich verbessern können.

5 Refactoring mit Generativer AI

5.1 Ausgangscode

Der ursprüngliche Java-Code enthielt mehrere Aufgaben innerhalb einer einzigen Methode.

```
public class TaskProcessorBefore {  
  
    public static void processTask(String taskName, String priority) {  
        System.out.println("Bearbeite Aufgabe: " + taskName);  
  
        if (priority.equals("hoch")) {  
            System.out.println("Hohe Priorität");  
        }  
  
        System.out.println("Status speichern");  
    }  
}
```

5.2 Refactoring durch die Generative AI

Für diesen Teil wurde ChatGPT als Generative AI verwendet.
Die Generative AI schlug vor, die Prioritätsprüfung in eine separate Methode auszulagern.

```
public class TaskProcessorAfter {  
  
    public static void processTask(String taskName, String priority) {  
        System.out.println("Bearbeite Aufgabe: " + taskName);  
        checkPriority(priority);  
        System.out.println("Status speichern");  
    }  
  
    private static void checkPriority(String priority) {  
        if (priority.equals("hoch")) {  
            System.out.println("Hohe Priorität");  
        }  
    }  
}
```

Das vorgeschlagene Refactoring entspricht dem Fowler-Refactoring „Extract Function“. In Java wird dieses Refactoring häufig als „Extract Method“ bezeichnet.

5.3 Erfahrungen

Die Generative AI erkannte die Möglichkeit zur Auslagerung von Code korrekt und schlug automatisch eine separate Methode vor.

Das Ergebnis war übersichtlicher und leichter verständlich als der ursprüngliche Code. Das Refactoring funktionierte bereits beim ersten Versuch und erforderte keine größere Nachbearbeitung.

Besonders hilfreich war, dass die AI den eigentlichen Zweck des Refactorings nachvollziehen konnte und nicht lediglich Variablen oder Methodennamen umbenannt hat.

5.4 Bewertung

Für einfache bis mittlere Refactorings kann Generative AI eine sinnvolle Unterstützung darstellen. Dennoch sollte das Ergebnis stets geprüft werden, da die Verantwortung für die Codequalität weiterhin beim Entwickler liegt.

6 Fazit

Im Rahmen dieser Einsendeaufgabe konnte ein erster praktischer Einblick in das Thema Refactoring gewonnen werden. Die Analyse ausgewählter Refactorings aus dem Katalog von Martin Fowler zeigte, dass Refactoring einen wichtigen Beitrag zur Verbesserung von Lesbarkeit, Wartbarkeit und Struktur von Software leisten kann.

Besonders überzeugend erschienen die Refactorings „Extract Function“ und „Extract Class“, da deren Nutzen unmittelbar nachvollziehbar ist und sie bereits mit geringem Aufwand zu einer besseren Codequalität beitragen können.

Durch das eigene Refactoring-Beispiel wurde deutlich, wie Verantwortlichkeiten innerhalb von Methoden klarer getrennt werden können. Die praktische Umsetzung verdeutlichte die Vorteile einer sauberen Strukturierung von Quellcode.

Darüber hinaus wurde untersucht, wie Generative AI bei Refactoring-Aufgaben unterstützen kann. Die AI konnte sinnvolle Verbesserungsvorschläge liefern und ein passendes Refactoring identifizieren. Gleichzeitig zeigte sich, dass die Ergebnisse weiterhin fachlich überprüft werden müssen.

Insgesamt hat die Bearbeitung der Aufgabe gezeigt, dass Refactoring ein wichtiger Bestandteil professioneller Softwareentwicklung ist und sowohl manuell als auch mit Unterstützung moderner Werkzeuge effektiv durchgeführt werden kann.